

Caching Process

Architecture, Strategies & Implementation Guide

A comprehensive technical reference covering cache architecture, placement tiers, read/write strategies, eviction and invalidation policies, distributed cache coherence, common failure modes, and production best practices.

Technical Reference Document

Prepared: July 2026

Table of Contents

1	Introduction to Caching
2	Why Caching Matters — The Performance Case
3	Core Cache Architecture & Request Flow
4	Cache Placement Tiers
5	Caching Strategies (Read & Write Patterns)
6	Cache Eviction Policies
7	Cache Invalidation
8	Distributed Caching, Partitioning & Consistency
9	Common Caching Technologies
10	Failure Modes & Mitigation
11	Key Metrics & Monitoring
12	Best Practices Checklist
13	Conclusion

1. Introduction to Caching

A **cache** is a high-speed storage layer that holds a subset of data, typically transient in nature, so that future requests for that data can be served faster than fetching it from its primary, slower storage location (a database, a remote API, disk, or a computation-heavy process). Caching is one of the most impactful and universally applicable optimization techniques in software systems — it trades a small amount of memory (or fast storage) for a large reduction in latency and load on backend systems.

The general principle behind caching is the **locality of reference**: applications tend to access the same data, or data close to previously accessed data, repeatedly over short windows of time. Caching exploits two forms of this locality:

- **Temporal locality** — data accessed recently is likely to be accessed again soon (e.g., a trending product page).
- **Spatial locality** — data near recently accessed data is likely to be accessed soon (e.g., adjacent rows, related API fields).

Caches exist at nearly every layer of a modern system — inside the CPU (L1/L2/L3 caches), the operating system (page cache), the browser, the network (CDN edge nodes and DNS resolvers), the application tier (in-process or distributed object caches), and the database (query and buffer caches). This document focuses primarily on the caching process as applied to web and distributed application architectures, mirroring the structure used for load balancer documentation, and works progressively from architecture, to strategy, to operations.

2. Why Caching Matters — The Performance Case

Every cache exists to solve one or more of the following problems:

- **Latency reduction** — memory access (nanoseconds to low microseconds) is orders of magnitude faster than disk I/O, a database query, or a network round trip to a remote service (milliseconds to hundreds of milliseconds).
- **Backend load reduction** — serving repeat requests from cache avoids redundant computation or repeated database hits, protecting the origin system from overload.
- **Cost efficiency** — fewer database reads and fewer compute cycles translate directly into lower infrastructure spend, especially for pay-per-query or pay-per-request backends.
- **Improved availability** — a cache can continue serving stale-but-useful data during a transient backend outage (a pattern known as “stale-while-revalidate” or graceful degradation).
- **Throughput scaling** — caching allows a system to serve many more requests per second than the origin could handle alone, since cache hits are cheap to satisfy.

Storage Layer	Typical Access Latency
CPU register	< 1 nanosecond
L1/L2 CPU cache	1 – 10 nanoseconds
In-memory cache (RAM, e.g. Redis)	0.1 – 1 millisecond
SSD disk	0.1 – 0.5 millisecond
HDD disk	5 – 10 milliseconds
Remote database query	5 – 100+ milliseconds
Cross-region network call	50 – 300+ milliseconds

Table 2.1 — Representative latency order-of-magnitude by storage tier (illustrative, not benchmarked figures).

3. Core Cache Architecture & Request Flow

At its core, every cache implementation answers four questions: **where** is the cache positioned relative to the client and origin, **what** triggers a read or write to it, **how** is stale or excess data removed, and **how** is consistency maintained when the underlying data changes. The following flow illustrates the canonical request path for a read-oriented cache sitting in front of a database.

```
CLIENT REQUEST FLOW (Cache-Aside Pattern)
-----
1. Client -> Application Server: GET /product/1024
2. App Server -> Cache: GET key='product:1024'
|
|-- HIT --> Cache returns cached value
| App Server -> Client: 200 OK (fast path)
|
'-- MISS --> App Server -> Database: SELECT * FROM products WHERE id=1024
Database -> App Server: row data
App Server -> Cache: SET key='product:1024' value=row TTL=300s
App Server -> Client: 200 OK (slow path, now cached)
```

Figure 3.1 — Cache-aside read path: the application mediates between cache and origin store.

Two outcomes define cache performance: a **cache hit** (the requested data is found in the cache and returned immediately) and a **cache miss** (the data is absent, forcing a fetch from the origin, usually followed by populating the cache for subsequent requests). The ratio of hits to total requests is the **cache hit ratio**, the single most important health metric for any cache layer.

3.1 Key Components

- **Cache store** — the actual data structure holding entries (in-memory hash map, SSD-backed store, or a distributed key-value cluster).
- **Key/value or key/object model** — most caches are addressed by a unique key (e.g., a URL, a composite of table+ID, or a query hash) mapping to a serialized value.
- **TTL (Time-To-Live) engine** — tracks expiration timestamps and removes or refreshes stale entries.
- **Eviction manager** — enforces the configured eviction policy once the cache reaches a size or memory limit.
- **Invalidation hooks** — logic (explicit calls, pub/sub events, or write-path triggers) that removes or updates cache entries when underlying data changes.

4. Cache Placement Tiers

Caching is typically layered — a single request may pass through several cache tiers before reaching the origin, with each layer absorbing a portion of traffic.

Tier	Location	Typical Use
Client / Browser cache	End-user device	Static assets (JS/CSS/images), HTTP cache headers (Cache-Control)
CDN / Edge cache	Geographically distributed PoPs	Static content, cacheable API responses, video segments
DNS cache	Resolver / OS / router	Domain name to IP resolution results
Reverse proxy cache	In front of app servers (e.g., Varnish, Nginx)	Full HTTP response caching, offloading app servers
Application / in-process cache	Inside the app runtime (e.g., local hash map, Off-heap)	Application-specific computed values, per-instance memoization
Distributed cache	Dedicated cache cluster (e.g., Redis, Memcached)	Shared session data, query results, rate-limit counters
Database cache	DB engine buffer pool / query cache	Frequently accessed pages, execution plans

Table 4.1 — Cache tiers ordered from closest-to-client to closest-to-origin.

As a request travels further from the client toward the origin, each tier that produces a hit prevents all subsequent, more expensive tiers from being invoked. This is why edge/CDN caching is prioritized for static or semi-static content, while distributed in-memory caches are reserved for dynamic, per-user, or frequently changing data that cannot be cached at the edge.

5. Caching Strategies (Read & Write Patterns)

How an application reads from and writes to a cache relative to the origin store defines its **caching strategy**. The choice affects consistency guarantees, latency on writes, and resilience to cache failure.

5.1 Cache-Aside (Lazy Loading)

The application is responsible for checking the cache first; on a miss, it reads from the origin and populates the cache. This is the most common pattern because it caches only what is actually requested and tolerates cache outages gracefully (the app simply falls back to the origin).

- **Pros:** only requested data is cached; resilient to cache failures; simple to reason about.
- **Cons:** first request after a miss is always slow ("cold" penalty); risk of stale data if invalidation is missed.

5.2 Read-Through

The cache itself sits transparently in front of the origin and is responsible for loading data on a miss, rather than the application. The application only ever talks to the cache. This centralizes loading logic but requires the cache layer to support pluggable loaders.

5.3 Write-Through

Every write goes to the cache and the origin store synchronously (cache is updated as part of the write transaction). This keeps the cache always consistent with the origin at the cost of added write latency, since both writes must succeed.

5.4 Write-Behind (Write-Back)

Writes are applied to the cache immediately and asynchronously flushed to the origin store later, often batched. This yields very low write latency and can smooth out write bursts, but introduces a risk window where data exists only in the cache and could be lost if the cache fails before the flush completes.

5.5 Write-Around

Writes go directly to the origin store, bypassing the cache. The cache is only populated on a subsequent read (cache-aside style). This avoids filling the cache with data that may never be read again, at the cost of a guaranteed miss on the first read after a write.

Strategy	Write Path	Consistency	Best Fit
Cache-aside	App writes DB only; cache invalidated on read (depends on invalidation)	Eventual (depends on invalidation)	General-purpose read-heavy workloads
Read-through	Cache loader fetches from origin on miss	Eventual	Simplifying app-side loading logic
Write-through	App writes cache + DB synchronously	Strong	Data that must never be stale (pricing, inventory)
Write-behind	App writes cache; DB updated asynchronously	Weak (durability risk)	High write throughput, tolerant of brief loss
Write-around	App writes DB only; cache untouched	Eventual, cold on first read	Write-heavy, rarely re-read data

Table 5.1 — Comparison of core caching strategies.

6. Cache Eviction Policies

Caches have finite capacity. When full, an **eviction policy** determines which entries are removed to make room for new ones. The right policy depends on the access pattern of the workload.

Policy	Mechanism	Notes
LRU (Least Recently Used)	Evicts the entry that has gone unused the longest	Most widely used default; good general-purpose behavior
LFU (Least Frequently Used)	Evicts the entry with the lowest access count	Favors long-term popularity over recency; needs counter
FIFO (First In, First Out)	Evicts the oldest inserted entry regardless of use	Simple, cheap, but ignores access patterns
MRU (Most Recently Used)	Evicts the most recently used entry	Useful for cyclic scan patterns (e.g., full table scans)
Random Replacement (RR)	Evicts a randomly chosen entry	Very low overhead; acceptable when access is near-uniform
TTL-based Expiry	Removes entries after a fixed time-to-live regardless of use	Used alongside another policy to bound staleness
ARC (Adaptive Replacement Cache)	Dynamically balances recency and frequency	More complex; adapts to changing workloads

Table 6.1 — Common eviction policies and their trade-offs.

In production systems, eviction policy and TTL are usually combined: TTL bounds the maximum staleness of any entry, while the eviction policy (commonly LRU or an LRU-approximation) governs which entries are dropped early under memory pressure.

7. Cache Invalidation

Phil Karlton's well-known observation — “there are only two hard things in computer science: cache invalidation and naming things” — reflects how difficult it is to guarantee a cache never serves stale data while still gaining the performance benefit of caching. The main invalidation approaches are:

- **TTL expiration** — the simplest approach; entries automatically expire after a fixed duration. Staleness is bounded but not eliminated.
- **Explicit invalidation (active)** — the write path explicitly deletes or updates the corresponding cache key immediately after a data change.
- **Event-driven invalidation** — a change data capture (CDC) stream or pub/sub message (e.g., from a database binlog or a message queue) triggers cache eviction asynchronously across all cache nodes.
- **Versioned / generational keys** — instead of deleting stale entries, the cache key itself encodes a version (e.g., `product:1024:v7`), so old versions simply age out via the eviction policy without explicit deletes.
- **Cache stampede protection on invalidation** — combined with locking or request coalescing so that when a hot key is invalidated, only one request repopulates it rather than a burst of concurrent origin calls.

7.1 Invalidation Scope

- **Single-key invalidation** — precise, low-cost, used when the affected entity is known (e.g., updating one product).
 - **Tag/group invalidation** — invalidates all entries associated with a tag (e.g., all cached pages referencing a given category) when a shared dependency changes.
 - **Full flush** — clears the entire cache; a blunt but sometimes necessary tool after a schema change or major data migration.
-

8. Distributed Caching, Partitioning & Consistency

As traffic and data volume grow beyond what a single cache node can hold or serve, caches are distributed across multiple nodes. This introduces the need for **partitioning** (deciding which node owns which keys) and **consistency** (keeping replicated or partitioned data coherent).

8.1 Partitioning (Sharding) Strategies

- **Modulo hashing** — $\text{node} = \text{hash}(\text{key}) \% N$; simple but requires a near-total remap when N changes, causing a cache-wide miss storm.
- **Consistent hashing** — maps both nodes and keys onto a hash ring; adding or removing a node only remaps a small fraction of keys, minimizing disruption.
- **Rendezvous (highest random weight) hashing** — each key independently selects its owning node by weight comparison, avoiding a shared ring structure.

8.2 Replication Models

- **Primary-replica replication** — writes go to a primary node and are asynchronously propagated to replicas, which serve reads and provide failover.
- **Peer-to-peer replication** — all nodes accept writes and gossip changes to each other; higher availability, more complex conflict resolution.

8.3 Cache Coherence

In multi-node caches, **coherence** refers to ensuring that reads from any node reflect the most recent write, or at least converge to it within a bounded time. Distributed caches typically accept **eventual consistency** in exchange for availability and low latency (per the CAP theorem), using techniques such as version vectors, last-write-wins timestamps, or invalidation broadcasts to converge state across nodes.

DISTRIBUTED CACHE WRITE PROPAGATION (simplified)

```
App Server -> Cache Node A (owner of key 'session:88'): SET session:88 = {...}
Cache Node A -> Cache Node B, C (replicas): replicate(session:88, value, version=42)
Cache Node B/C: apply if incoming version > local version (last-write-wins)
```

Figure 8.1 — Simplified replication and conflict-resolution flow across cache replicas.

9. Common Caching Technologies

Technology	Category	Notable Characteristics
Redis	In-memory data store / distributed cache	Rich data structures, persistence options, pub/sub, clustering
Memcached	In-memory key-value cache	Simple, multithreaded, purely volatile (no persistence)
Varnish	HTTP reverse-proxy cache	Full-page HTTP caching with a configurable VCL policy language
Nginx caching	Reverse-proxy / edge cache	Proxy-cache directives for HTTP response caching
CDN (e.g., edge networks)	Edge / geo-distributed cache	Caches static and semi-dynamic content close to end users
Ehcache / Caffeine	In-process (JVM) application cache	Local, extremely low latency, no network hop
Database buffer pool	Storage-engine cache	Caches data pages / query plans inside the DB engine itself

Table 9.1 — Representative caching technologies by category.

10. Failure Modes & Mitigation

10.1 Cache Stampede (Thundering Herd)

Occurs when a popular key expires and many concurrent requests simultaneously miss the cache, all hammering the origin store at once. **Mitigation:** request coalescing / single-flight locks (only one request repopulates, others wait), early/probabilistic TTL refresh (e.g., XFetch), or staggered TTL jitter so keys do not expire in unison.

10.2 Cache Penetration

Repeated queries for keys that do not exist in the origin (e.g., an invalid ID) bypass the cache entirely every time, since nothing is ever cached for a miss. **Mitigation:** cache negative results with a short TTL, or use a probabilistic filter (e.g., a Bloom filter) to reject known-nonexistent keys before hitting the origin.

10.3 Cache Avalanche

A large number of cache entries expire at the same moment (or an entire cache node/cluster fails), causing a sudden surge of traffic to the origin. **Mitigation:** randomized TTL jitter, multi-node redundancy, and circuit breakers or rate limiting in front of the origin store.

10.4 Hot Key / Hot Partition

A single extremely popular key concentrates traffic onto one cache node or shard, creating a bottleneck even though the overall cluster has capacity. **Mitigation:** local (in-process) caching of hot keys in front of the

distributed cache, key sharding/replication of the hot value across multiple nodes, or client-side read replicas.

10.5 Stale Data

Data changes at the origin but the cache is not invalidated, causing clients to see outdated values. **Mitigation:** reliable invalidation hooks on the write path, shorter TTLs for volatile data, and versioned cache keys.

11. Key Metrics & Monitoring

Metric	Definition	Why It Matters
Hit ratio	hits / (hits + misses)	Primary indicator of cache effectiveness
Miss ratio	misses / (hits + misses)	Inverse of hit ratio; high values signal poor sizing or key design
Eviction rate	Entries evicted per unit time due to capacity	High rate suggests the cache is undersized for the working set
Latency (p50/p95/p99)	Time to serve a cache read/write	Detects degradation, saturation, or network issues
Memory utilization	Used memory vs. allocated capacity	Prevents OOM evictions and informs capacity planning
Stale read rate	Reads served with outdated data (if measurable)	Surfaces invalidation gaps
Replication lag	Delay between primary write and replica visibility	Bounds the consistency window in distributed caches

Table 11.1 — Metrics to monitor for a healthy cache layer.

A healthy, well-sized cache for a read-heavy workload commonly sustains a hit ratio in the 90%+ range; sustained drops below this threshold typically indicate an undersized cache, a TTL that is too short, a poor key design causing fragmentation, or a shift in traffic pattern.

12. Best Practices Checklist

- Cache the results of expensive operations (DB joins, external API calls, heavy computation) rather than raw, cheap-to-fetch data.
- Choose TTLs deliberately based on how tolerant the data is to staleness — shorter for volatile data, longer for near-static data.
- Design cache keys to be specific, collision-free, and inclusive of all parameters that affect the cached value (e.g., locale, user segment, API version).
- Always have a fallback path to the origin store if the cache is unavailable — never let a cache outage become a full outage.
- Apply jitter to TTLs to avoid synchronized mass expiration (cache avalanche).

- Protect hot keys with request coalescing / locking to prevent stampedes on expiry.
 - Cache negative results (with a short TTL) to defend against cache penetration from invalid lookups.
 - Monitor hit ratio, memory pressure, and eviction rate continuously, and alert on sustained degradation.
 - Use consistent hashing (or an equivalent) for distributed caches to minimize remapping churn when scaling the cluster.
 - Prefer explicit or event-driven invalidation over relying on TTL alone for data with correctness requirements.
 - Right-size cache capacity to the actual working set — oversized caches waste memory, undersized caches evict too aggressively.
 - Version cache keys or schemas when the shape of cached data changes, to avoid deserialization errors on deploy.
-

13. Conclusion

Caching is a layered, system-wide discipline rather than a single component — effective caching requires deliberate choices at every level: where the cache sits, which read/write strategy governs it, how entries are evicted and invalidated, and how consistency is maintained once the cache spans multiple nodes. When designed and monitored carefully, caching delivers dramatic latency reductions and backend load relief; when neglected, it becomes a source of subtle correctness bugs (stale data) and cascading failures (stampedes, avalanches). The strategies, policies, and metrics outlined in this document provide a practical foundation for designing, operating, and troubleshooting a caching layer in a production system.

— End of Document —